

LIST OF EXPERIMENTS

Set 11: Product Category Analysis

1. Generate a funnel chart to analyze the sales conversion process for each product category. Label the stages and title the chart.

Code:

```
# Load required libraries
library(ggplot2)
library(dplyr)
library(knitr)

# Create the product categories dataset
category_data <- data.frame(
  Category = c("Electronics", "Appliances", "Clothing"),
  Sales = c(50000, 40000, 35000)
)

# Sort by Sales descending (required for funnel charts)
category_data <- category_data %>% arrange(desc(Sales))

print(category_data)

# 1. Funnel Chart: Sales Conversion / Hierarchy by Category
ggplot(category_data, aes(x = reorder(Category, Sales), y = Sales, fill = Category)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_text(
    aes(label = paste0(Category, "\n$", format(Sales, big.mark = ","))),
    hjust = 1.1,
    color = "white",
    fontface = "bold",
```

```

    size = 3.8
  ) +
  coord_flip() +
  scale_y_continuous(labels = scales::dollar_format()) +
  scale_fill_brewer(palette = "YlOrRd", direction = -1) +
  labs(
    title = "Product Category Sales Funnel",
    subtitle = "Hierarchical sales progression ordered by top revenue drivers",
    x = "Category Stage",
    y = "Sales Volume ($)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    panel.grid.major.y = element_blank()
  )

```

Output:



2. Build a table to display the sales data for each product category. Label the table elements.

Code:

```
# Load required libraries
```

```
library(ggplot2)
```

```
library(dplyr)
```

```
library(knitr)
```

```
# Create the product categories dataset
```

```
category_data <- data.frame(
```

```

Category = c("Electronics", "Appliances", "Clothing"),
Sales = c(50000, 40000, 35000)
)

```

```

# Sort by Sales descending (required for funnel charts)
category_data <- category_data %>% arrange(desc(Sales))

```

```

print(category_data)

```

2. Formatted Table Output

```

category_table <- category_data %>%
  mutate(
    `Sales Amount` = paste0("$", format(Sales, big.mark = ",")),
    `Share of Total` = paste0(round((Sales / sum(Sales)) * 100, 1), "%")
  ) %>%
  rename(`Product Category` = Category) %>%
  select(-Sales)

```

Display formatted table using knitr

```

kable(category_table, caption = "Product Category Sales Summary Table", align = "c")

```

Output:

```

Table: Product Category Sales Summary Table

| Product Category | Sales Amount | Share of Total |
|:-----:|:-----:|:-----:|
| Electronics | $50,000 | 40% |
| Appliances | $40,000 | 32% |
| Clothing | $35,000 | 28% |
> |

```

3. Develop a Tableau dashboard combining the pie chart, funnel chart, and the table for interactive

exploration of product category data.

Code:

```
# Load required libraries
```

```
library(ggplot2)
```

```
library(dplyr)
```

```
library(knitr)
```

```
# Create the product categories dataset
```

```
category_data <- data.frame(  
  Category = c("Electronics", "Appliances", "Clothing"),  
  Sales = c(50000, 40000, 35000)  
)
```

```
# Sort by Sales descending (required for funnel charts)
```

```
category_data <- category_data %>% arrange(desc(Sales))
```

```
print(category_data)
```

```
# =====
```

```
# Set 11 — Task 3: Tableau Data Preparation Script
```

```
# =====
```

```
library(dplyr)
```

```
# 1. Recreate dataset
```

```
category_data <- data.frame(  
  Category = c("Electronics", "Appliances", "Clothing"),  
  Sales = c(50000, 40000, 35000)  
) %>% arrange(desc(Sales))
```

```
# 2. Add derived percentage column for Tableau
```

```
category_data <- category_data %>%
```

```
mutate(
  Sales_Percent = round((Sales / sum(Sales)) * 100, 2)
)

# 3. Export to CSV for direct import into Tableau
write.csv(category_data, "Product_Categories_Tableau.csv", row.names = FALSE)

print("File 'Product_Categories_Tableau.csv' successfully created!")
print(category_data)
```

Output:

```

  Category Sales Sales_Percent
1 Electronics 50000           40
2 Appliances 40000           32
3 Clothing 35000            28
> |
```

4. Create a pie chart to represent the distribution of sales across product categories. Include labels.

Code:

```
# =====

# Set 11 — Task 4: Standalone Pie Chart Code

# =====

library(ggplot2)
library(dplyr)

# 1. Dataset
category_data <- data.frame(
  Category = c("Electronics", "Appliances", "Clothing"),
  Sales = c(50000, 40000, 35000)
)

# 2. Calculate percentages and create text labels
```

```

sales_pie_data <- category_data %>%
  mutate(
    Percentage = Sales / sum(Sales) * 100,
    Label = paste0(Category, "\n$", format(Sales, big.mark = ","), "\n(", round(Percentage, 1),
"%%)")
  )

```

3. Build and print the Pie Chart

```

sales_pie_chart <- ggplot(sales_pie_data, aes(x = "", y = Sales, fill = Category)) +
  geom_col(width = 1, color = "white") +
  coord_polar(theta = "y") +
  geom_text(
    aes(label = Label),
    position = position_stack(vjust = 0.5),
    color = "white",
    fontface = "bold",
    size = 3.8
  ) +
  scale_fill_brewer(palette = "Set1") +
  labs(
    title = "Distribution of Sales Across Product Categories",
    subtitle = "Percentage share of revenue per category",
    fill = "Product Category"
  ) +
  theme_void(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold", size = 14, hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
  )

```

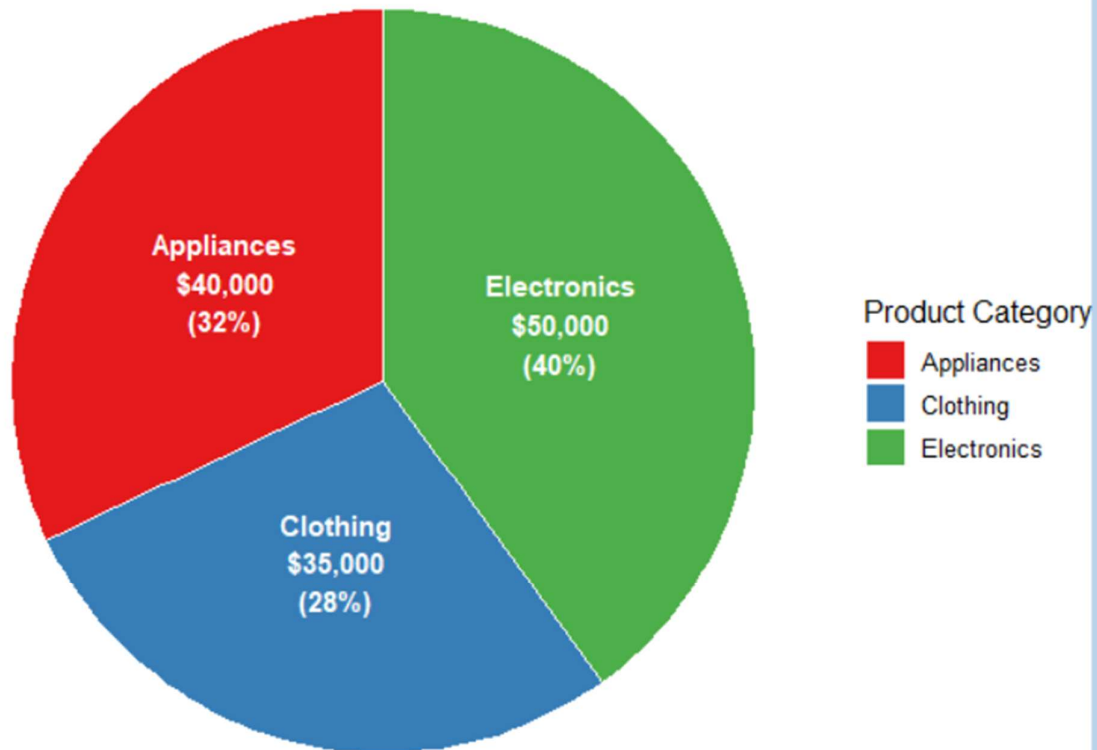
Render plot cleanly on a new line

```
print(sales_pie_chart)
```

Output:

Distribution of Sales Across Product Categories

Percentage share of revenue per category



Set 12: Website Traffic

1. Create a line chart to visualize the trend in daily page views over time. Label the axes and title the

chart.

Code:

```
# Load required libraries
```

```
library(ggplot2)
```

```
library(dplyr)
```

```
library(knitr)
```



```

# Create the website traffic dataset

traffic_data <- data.frame(
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03")),
  Page_Views = c(1500, 1600, 1400),
  CTR = c(2.3, 2.7, 2.0)
)

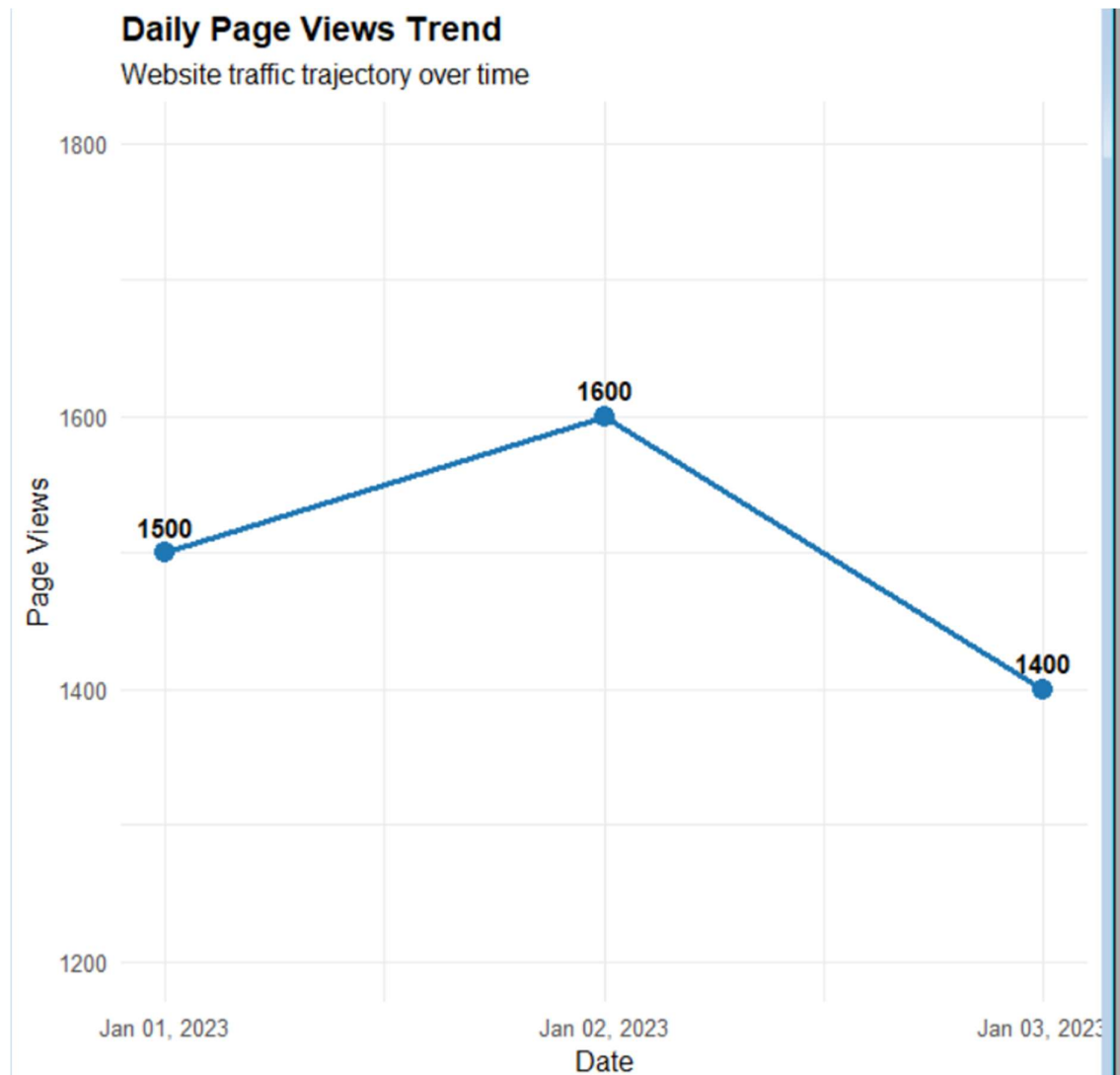
print(traffic_data)

# 1. Line Chart: Daily Page Views Trend

ggplot(traffic_data, aes(x = Date, y = Page_Views)) +
  geom_line(color = "#1F77B4", size = 1.2) +
  geom_point(color = "#1F77B4", size = 4) +
  geom_text(aes(label = Page_Views), vjust = -0.8, fontface = "bold", size = 3.8) +
  scale_x_date(date_labels = "%b %d, %Y", date_breaks = "1 day") +
  scale_y_continuous(limits = c(1200, 1800)) +
  labs(
    title = "Daily Page Views Trend",
    subtitle = "Website traffic trajectory over time",
    x = "Date",
    y = "Page Views"
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold", size = 14))

```

Output:



2. Generate a bar chart showing the top N days with the highest click-through rates.

Label the chart

elements.

Code:

```
# Load required libraries
```

```
library(ggplot2)
```

```
library(dplyr)
```

```
library(knitr)
```

```
# Create the website traffic dataset
```

```
traffic_data <- data.frame(
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03")),
  Page_Views = c(1500, 1600, 1400),
  CTR = c(2.3, 2.7, 2.0)
)
```

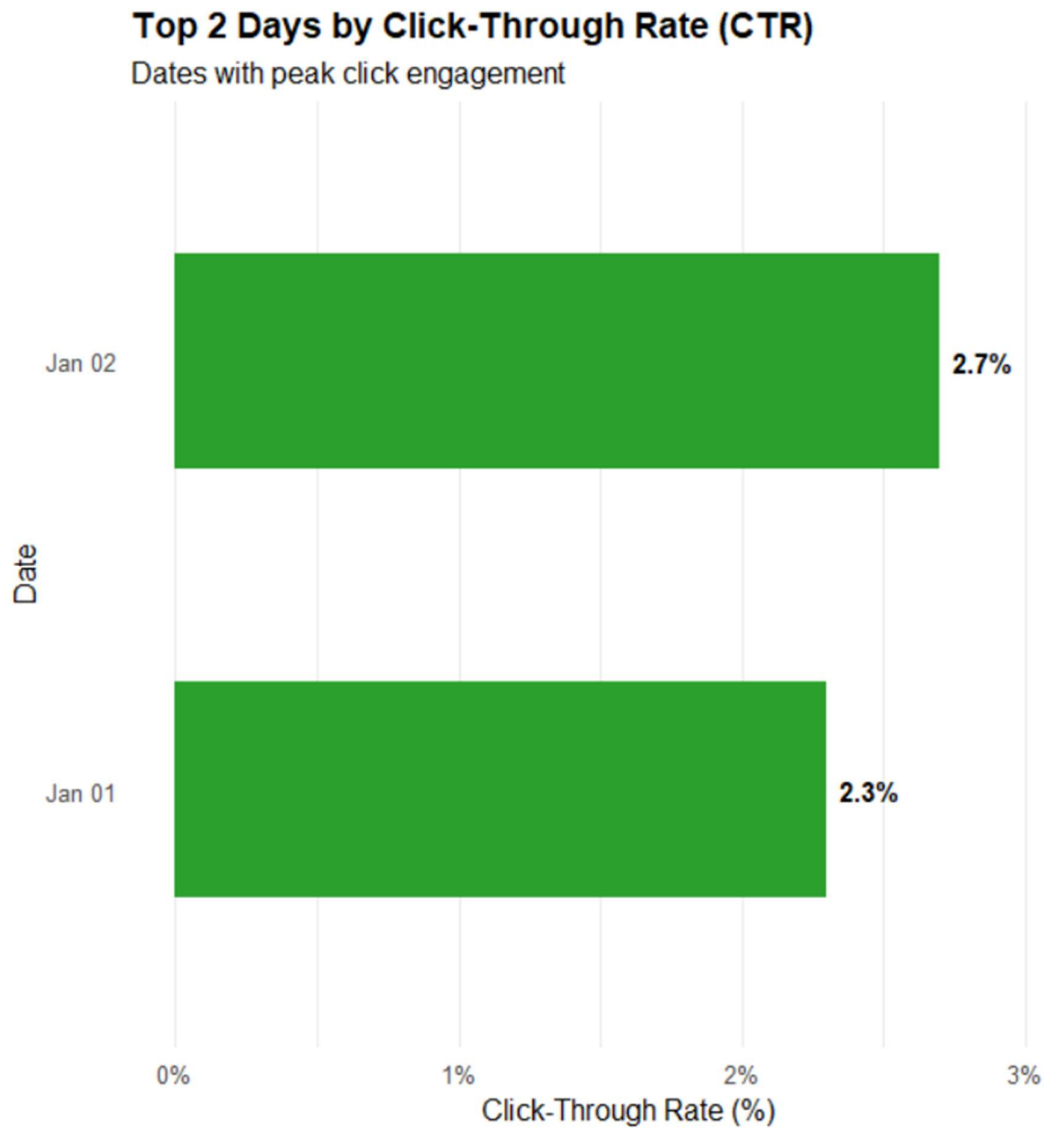
```
print(traffic_data)
```

2. Bar Chart: Top N Days by Click-Through Rate

```
top_n_days <- 2 # Adjust N as needed
```

```
traffic_data %>%
  slice_max(order_by = CTR, n = top_n_days) %>%
  ggplot(aes(x = reorder(format(Date, "%b %d"), CTR), y = CTR)) +
  geom_col(fill = "#2CA02C", width = 0.5) +
  geom_text(aes(label = paste0(CTR, "%")), hjust = -0.2, fontface = "bold", size = 3.8) +
  coord_flip() +
  scale_y_continuous(limits = c(0, 3.2), labels = function(x) paste0(x, "%")) +
  labs(
    title = paste("Top", top_n_days, "Days by Click-Through Rate (CTR)"),
    subtitle = "Dates with peak click engagement",
    x = "Date",
    y = "Click-Through Rate (%)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    panel.grid.major.y = element_blank()
  )
```

Output:



3. Build a table to show the website traffic data for each day. Label the table elements.

Code:

```
# Load required libraries
```

```
library(ggplot2)
```

```
library(dplyr)
```

```
library(knitr)
```

```
# Create the website traffic dataset
```

```
traffic_data <- data.frame(
```

```
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03")),
```

```

Page_Views = c(1500, 1600, 1400),
CTR = c(2.3, 2.7, 2.0)
)

```

```

print(traffic_data)

```

3. Formatted Summary Table Output

```

traffic_table <- traffic_data %>%
  mutate(
    Date = format(Date, "%Y-%m-%d"),
    CTR = paste0(CTR, "%")
  ) %>%
  rename(
    `Date` = Date,
    `Page Views` = Page_Views,
    `Click-Through Rate` = CTR
  )

```

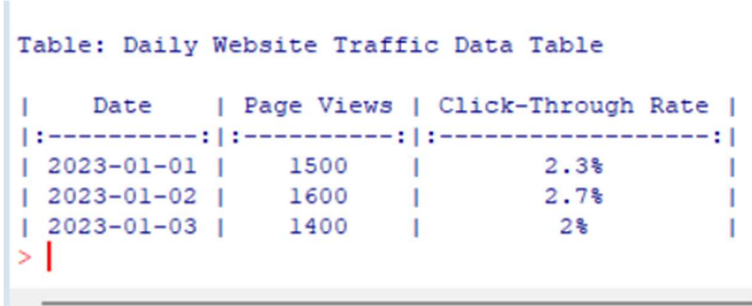
Display formatted table

```

kable(traffic_table, caption = "Daily Website Traffic Data Table", align = "c")

```

Output:



```

Table: Daily Website Traffic Data Table

```

Date	Page Views	Click-Through Rate
2023-01-01	1500	2.3%
2023-01-02	1600	2.7%
2023-01-03	1400	2%

4. Develop a Tableau dashboard combining the line chart, bar chart, and the table for interactive

exploration of website traffic data.

Code:

```
# =====  
# Set 12 — Task 4: Tableau Data Preparation Script  
# =====  
  
library(dplyr)  
  
# 1. Recreate dataset  
traffic_data <- data.frame(  
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03")),  
  Page_Views = c(1500, 1600, 1400),  
  CTR = c(2.3, 2.7, 2.0)  
)  
  
# 2. Add formatted percentage string column for Tableau labels  
traffic_data_tableau <- traffic_data %>%  
  mutate(  
    Date_Formatted = format(Date, "%Y-%m-%d"),  
    CTR_Percent = paste0(CTR, "%")  
  )  
  
# 3. Export to CSV file  
write.csv(traffic_data_tableau, "Website_Traffic_Tableau.csv", row.names = FALSE)  
  
print("File 'Website_Traffic_Tableau.csv' created successfully!")  
print(traffic_data_tableau)
```

Output:

```

      Date Page_VIEWS CTR Date_Formatted CTR_Percent
1 2023-01-01      1500 2.3    2023-01-01      2.3%
2 2023-01-02      1600 2.7    2023-01-02      2.7%
3 2023-01-03      1400 2.0    2023-01-03      2%
> |

```

Set 13: Geographic Data

1. Create a map chart to visualize the distribution of cities on a geographic map. Label the map

elements.

Code:

```

# =====

# TASK 1: MAP CHART

# =====

# 1. Install & Load Required Packages
if (!require("ggplot2")) install.packages("ggplot2")
if (!require("maps")) install.packages("maps")

library(ggplot2)
library(maps)

# 2. Define the Dataset
# (Coordinates added to place points on the map)
geo_data <- data.frame(
  City = c("City A", "City B", "City C"),
  Population = c(500000, 700000, 600000),
  Avg_Temp = c(75, 68, 80),
  Elevation = c(1000, 800, 1200),
  Lon = c(-95.36, -118.24, -87.62),
  Lat = c(29.76, 34.05, 41.87)

```

)

3. Load Offline Base Map

```
us_map <- map_data("usa")
```

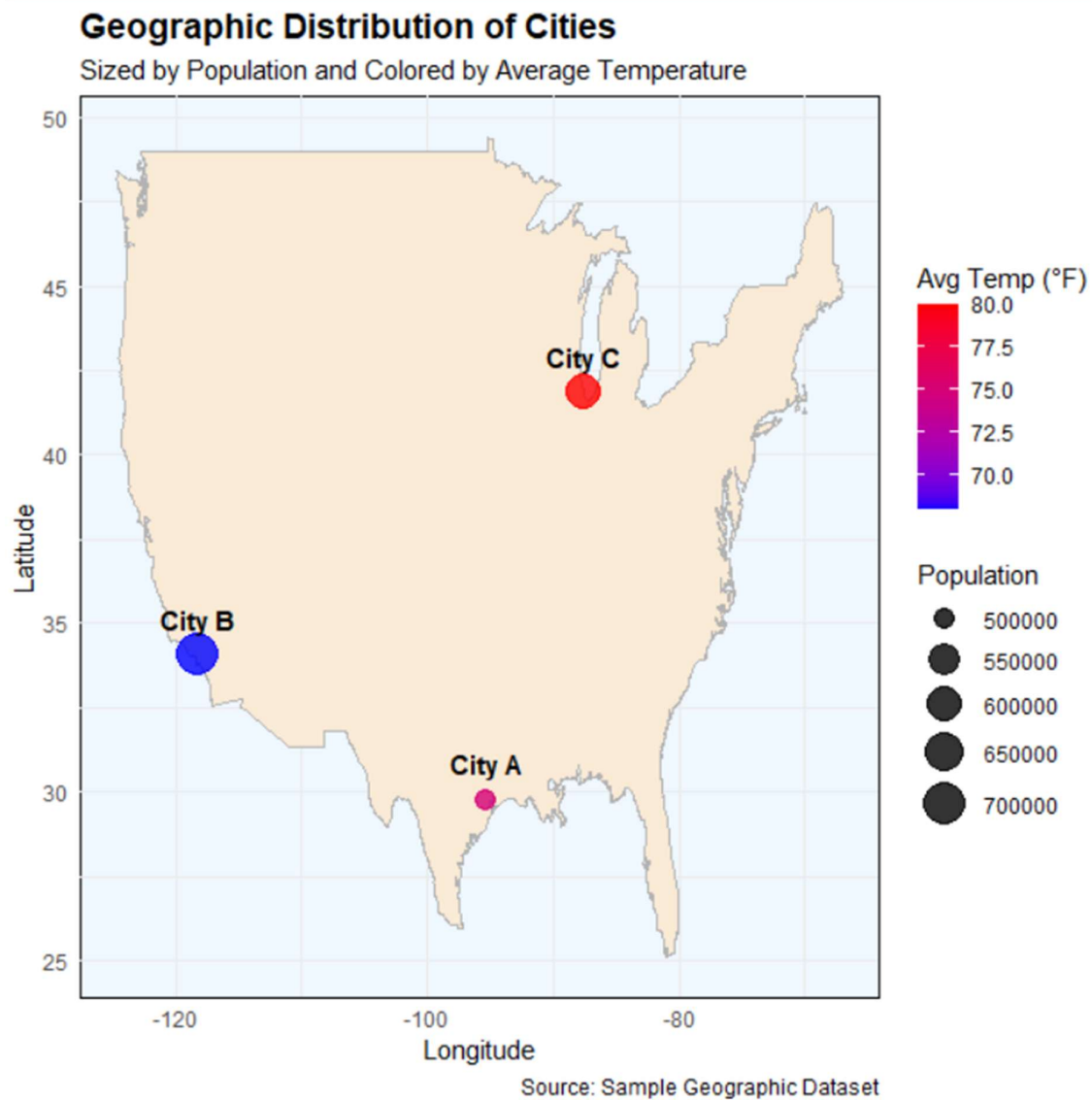
4. Create and Render the Map Chart

```
map_chart <- ggplot() +  
  geom_polygon(data = us_map, aes(x = long, y = lat, group = group),  
    fill = "antiquewhite", color = "gray70") +  
  geom_point(data = geo_data, aes(x = Lon, y = Lat, size = Population, color = Avg_Temp),  
    alpha = 0.8) +  
  geom_text(data = geo_data, aes(x = Lon, y = Lat, label = City),  
    vjust = -1.2, fontface = "bold") +  
  scale_color_gradient(low = "blue", high = "red", name = "Avg Temp (°F)") +  
  scale_size_continuous(range = c(4, 8), name = "Population") +  
  labs(  
    title = "Geographic Distribution of Cities",  
    subtitle = "Sized by Population and Colored by Average Temperature",  
    x = "Longitude",  
    y = "Latitude",  
    caption = "Source: Sample Geographic Dataset"  
  ) +  
  theme_minimal() +  
  theme(  
    panel.background = element_rect(fill = "aliceblue"),  
    plot.title = element_text(face = "bold", size = 14)  
  )
```

5. Display the Map


```
print(map_chart)
```

Output:



2. Generate a scatter plot to explore the relationship between average temperature and population

size. Explain any insights.

Code:

```
# =====  
# TASK 2: SCATTER PLOT  
# =====
```

1. Load Required Library

```
if (!require("ggplot2")) install.packages("ggplot2")
```

```
library(ggplot2)
```

2. Define the Dataset

```
geo_data <- data.frame(  
  City = c("City A", "City B", "City C"),  
  Population = c(500000, 700000, 600000),  
  Avg_Temp = c(75, 68, 80),  
  Elevation = c(1000, 800, 1200)  
)
```

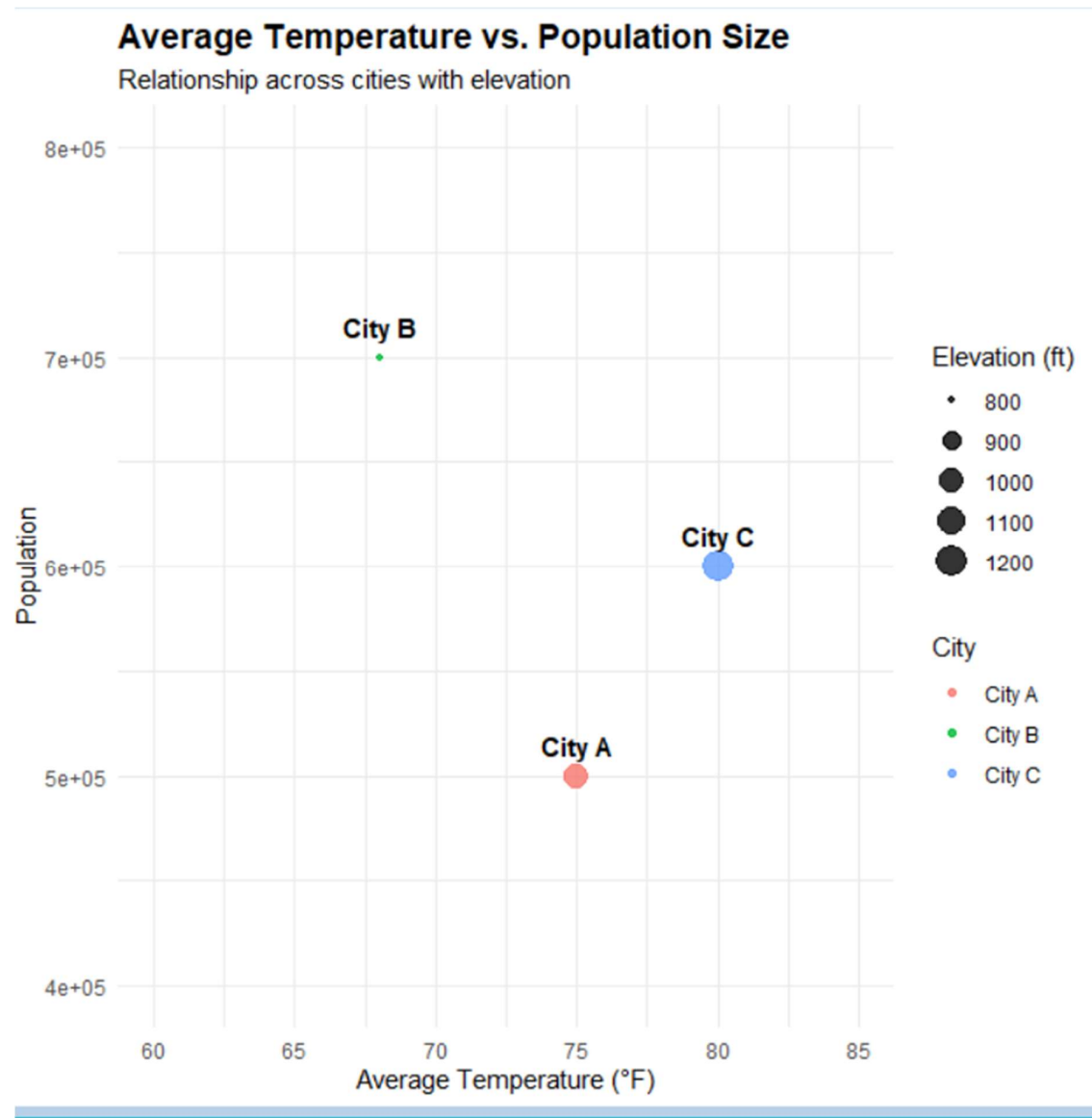
3. Create Scatter Plot

```
scatter_plot <- ggplot(geo_data, aes(x = Avg_Temp, y = Population)) +  
  geom_point(aes(color = City, size = Elevation), alpha = 0.8) +  
  geom_text(aes(label = City), vjust = -1, fontface = "bold") +  
  scale_y_continuous(limits = c(400000, 800000)) +  
  scale_x_continuous(limits = c(60, 85)) +  
  labs(  
    title = "Average Temperature vs. Population Size",  
    subtitle = "Relationship across cities with elevation",  
    x = "Average Temperature (°F)",  
    y = "Population",  
    size = "Elevation (ft)",  
    color = "City"  
  ) +  
  theme_minimal() +  
  theme(plot.title = element_text(face = "bold", size = 14))
```

4. Display the Plot

```
print(scatter_plot)
```

Output:



3. Build a table to display the geographic data for each city. Label the table elements.

Code:

```
# =====  
# TASK 3: DATA TABLE  
# =====
```

```

# 1. Load Required Library

if (!require("knitr")) install.packages("knitr")

library(knitr)


# 2. Define the Dataset

geo_data <- data.frame(
  City = c("City A", "City B", "City C"),
  Population = c(500000, 700000, 600000),
  Avg_Temp = c(75, 68, 80),
  Elevation = c(1000, 800, 1200)
)


# 3. Format Table Column Labels

table_data <- geo_data

colnames(table_data) <- c("City Name", "Population", "Avg Temperature (°F)", "Elevation (ft)")


# 4. Display Table in Console

kable(table_data, align = c("l", "r", "r", "r"), caption = "Table 1: Geographic Data Summary")

```

Output:

```

Table: Table 1: Geographic Data Summary

|City Name | Population| Avg Temperature (°F)| Elevation (ft)|
|:-----:|:-----:|:-----:|:-----:|
|City A    |      5e+05|           75|         1000|
|City B    |      7e+05|           68|           800|
|City C    |      6e+05|           80|         1200|
> |

```

Set 14:Survey Results

1. Create a stacked bar chart to display the distribution of answers for Question 1.

Label the chart

elements.

Code:

```
# =====  
# TASK 1: STACKED BAR CHART  
# =====  
  
# 1. Load Library  
if (!require("ggplot2")) install.packages("ggplot2")  
library(ggplot2)  
  
# 2. Define Dataset  
survey_data <- data.frame(  
  Respondent = c(1, 2, 3),  
  Q1 = c("A", "B", "C"),  
  Q2 = c("B", "A", "A"),  
  Q3 = c("C", "D", "B")  
)  
  
# 3. Create Frequency Table for Q1  
q1_df <- as.data.frame(table(Answer = survey_data$Q1))  
  
# 4. Build Stacked Bar Chart  
stacked_bar <- ggplot(q1_df, aes(x = "Question 1", y = Freq, fill = Answer)) +  
  geom_bar(stat = "identity", width = 0.4, color = "black") +  
  geom_text(aes(label = paste0(Answer, " (", Freq, ")")),  
    position = position_stack(vjust = 0.5), fontface = "bold", color = "white") +  
  scale_fill_manual(values = c("A" = "#4C72B0", "B" = "#55A868", "C" = "#C44E52")) +  
  labs(  
    title = "Distribution of Answers for Question 1",
```

```

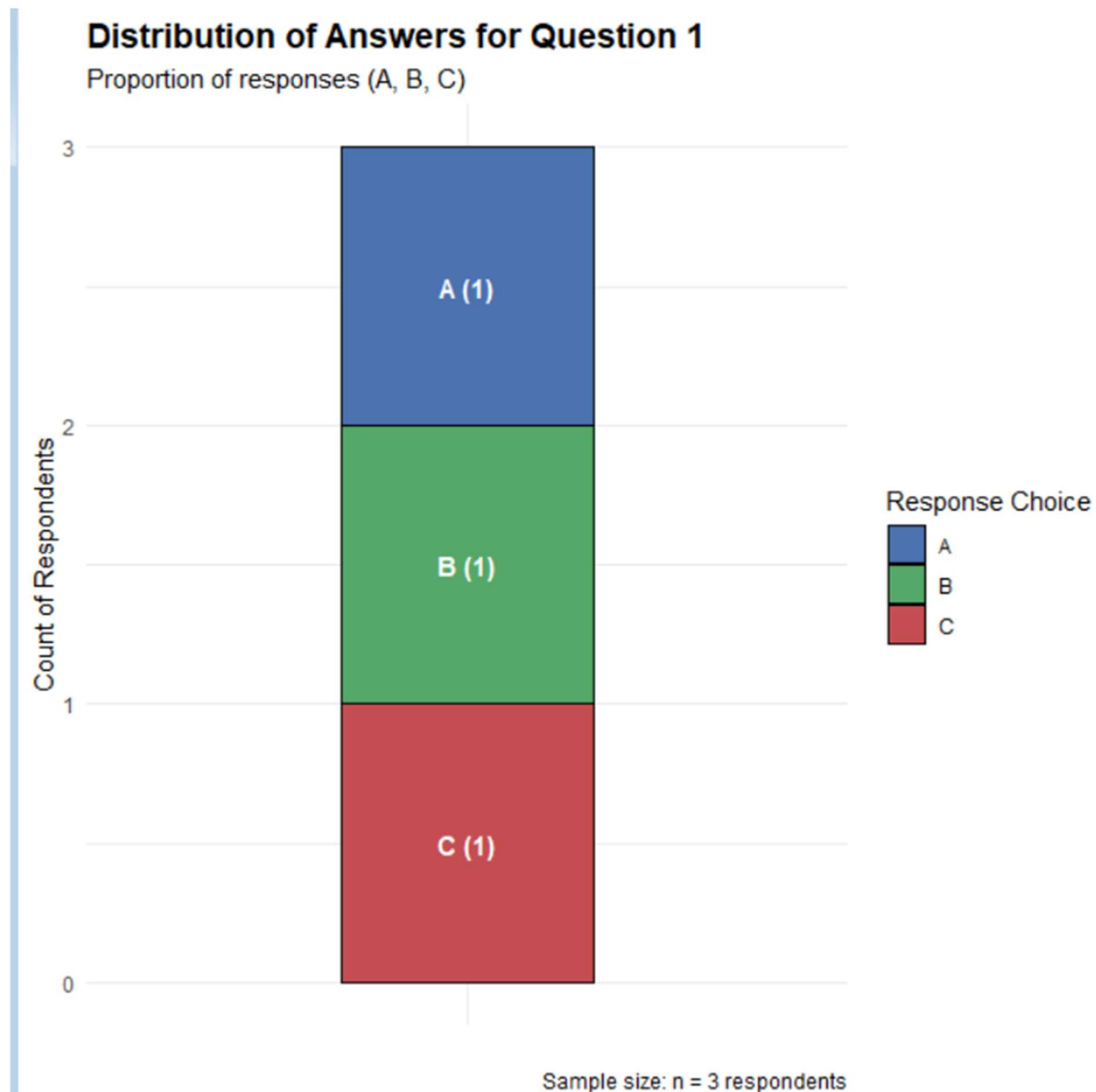
    subtitle = "Proportion of responses (A, B, C)",
    x = "",
    y = "Count of Respondents",
    fill = "Response Choice",
    caption = "Sample size: n = 3 respondents"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    axis.text.x = element_blank(),
    axis.ticks.x = element_blank()
  )

```

5. Display Chart

```
print(stacked_bar)
```

Output:



2. Generate a radar chart to visualize the overall pattern of responses across all three questions.

Code:

```
# =====
# TASK 2: RADAR CHART
# =====

# 1. Load Libraries
if (!require("fmsb")) install.packages("fmsb")
if (!require("tidyr")) install.packages("tidyr")
```

```
library(fmsb)
```

```
library(tidyr)
```

```
# 2. Reshape Data & Count Response Frequencies
```

```
survey_long <- pivot_longer(survey_data, cols = starts_with("Q"),  
                             names_to = "Question", values_to = "Answer")
```

```
# Cross-tabulate Frequency: Rows = Options (A, B, C, D), Columns = Questions
```

```
freq_table <- table(survey_long$Answer, survey_long$Question)  
radar_df <- as.data.frame.matrix(freq_table)
```

```
# 3. Format Data Frame for fmsb::radarchart
```

```
# fmsb requires the first row to be MAX limit and second row to be MIN limit
```

```
max_min <- data.frame(  
  Q1 = c(2, 0),  
  Q2 = c(2, 0),  
  Q3 = c(2, 0)  
)  
rownames(max_min) <- c("max", "min")
```

```
# Combine max/min limits with frequency data
```

```
radar_data <- rbind(max_min, radar_df)
```

```
# 4. Draw Radar Chart
```

```
# Set up layout and colors
```

```
colors_border <- c("#4C72B0", "#55A868", "#C44E52", "#8172B0")  
colors_in <- c(rgb(0.3,0.4,0.7,0.2), rgb(0.3,0.7,0.4,0.2),  
               rgb(0.8,0.3,0.3,0.2), rgb(0.5,0.4,0.7,0.2))
```



```

par(mar = c(1, 1, 2, 1)) # Set plot margins
radarchart(
  radar_data,
  axistype = 1,
  pcol = colors_border,
  pfc col = colors_in,
  plwd = 2,
  plty = 1,
  cglcol = "grey",
  cglty = 1,
  axislabcol = "grey",
  caxislabels = seq(0, 2, 1),
  cglwd = 0.8,
  vl cex = 1.1,
  title = "Response Frequency Pattern Across Questions"
)

```

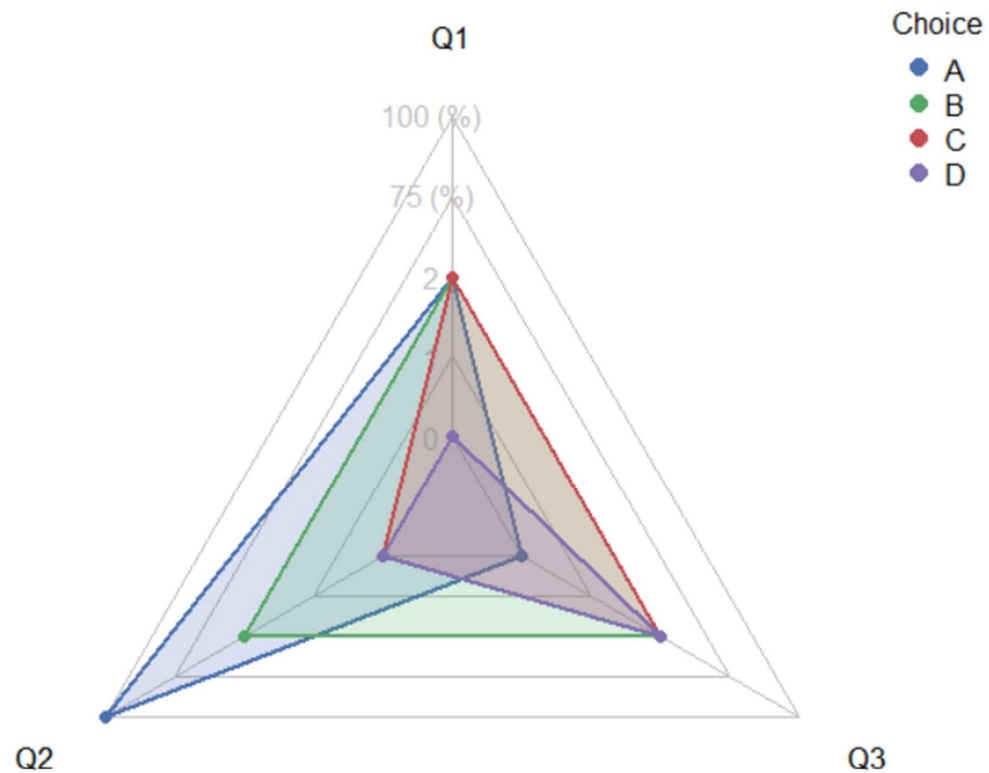
```

# Add Legend for Answer Choices (A, B, C, D)
legend(
  x = "topright",
  legend = rownames(radar_df),
  bty = "n",
  pch = 20,
  col = colors_border,
  text.col = "black",
  pt.cex = 2,
  title = "Choice"
)

```

Output:

Response Frequency Pattern Across Questions



3. Build a table to show the survey response data for each respondent. Label the table elements.

Code:

```
# =====  
# TASK 3: DATA TABLE  
# =====
```

```
# 1. Load Library
```

```
if (!require("knitr")) install.packages("knitr")
```

```
library(knitr)
```

```
# 2. Prepare Display Data Frame
```

```
table_display <- survey_data
```

```
colnames(table_display) <- c("Respondent ID", "Question 1", "Question 2", "Question 3")
```

```
# 3. Render Table
```

```
kable(  
  table_display,  
  align = c("c", "c", "c", "c"),  
  caption = "Table 1: Individual Survey Response Data"  
)
```

Output:

```
Table: Table 1: Individual Survey Response Data  
  
| Respondent ID | Question 1 | Question 2 | Question 3 |  
|:-----:|:-----:|:-----:|:-----:|  
|      1      |      A      |      B      |      C      |  
|      2      |      B      |      A      |      D      |  
|      3      |      C      |      A      |      B      |  
> |
```

Set 15: Product Sales Analysis

1. Create a grouped bar chart to visualize the sales of each product for the first quarter.

Label the

chart elements.

Code:

```
# =====  
# TASK 1: GROUPED BAR CHART  
# =====
```

```
# 1. Load Required Libraries
```

```
if(!require("ggplot2")) install.packages("ggplot2")
```

```
if(!require("tidyr")) install.packages("tidyr")
```

```
library(ggplot2)
```

```
library(tidyr)
```

```
# 2. Define Dataset
```

```
sales_data <- data.frame(  
  Product_ID = c(1, 2, 3),  
  Product_Name = c("Product A", "Product B", "Product C"),  
  January = c(2000, 1500, 1200),  
  February = c(2200, 1800, 1400),  
  March = c(2400, 1600, 1100)  
)
```

```
# 3. Reshape Data to Long Format for Grouped Charting
```

```
sales_long <- pivot_longer(  
  sales_data,  
  cols = c("January", "February", "March"),  
  names_to = "Month",  
  values_to = "Sales"  
)
```

```
# Lock month order chronologically
```

```
sales_long$Month <- factor(sales_long$Month, levels = c("January", "February", "March"))
```

```
# 4. Create Grouped Bar Chart
```

```
grouped_bar <- ggplot(sales_long, aes(x = Product_Name, y = Sales, fill = Month)) +  
  geom_bar(stat = "identity", position = position_dodge(width = 0.8), width = 0.7) +  
  geom_text(  
    aes(label = scales::comma(Sales)),  
    position = position_dodge(width = 0.8),  
    vjust = -0.5,
```

```

    size = 3.5,
    fontface = "bold"
  ) +
  scale_fill_manual(values = c("January" = "#2b5c8f", "February" = "#4682b4", "March" =
"#6baed6")) +
  scale_y_continuous(labels = scales::comma, limits = c(0, 3000)) +
  labs(
    title = "Q1 Product Sales Comparison",
    subtitle = "Monthly sales breakdown for Product A, B, and C",
    x = "Product Name",
    y = "Sales Revenue ($)",
    fill = "Month",
    caption = "Source: Monthly Product Sales Dataset"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    legend.position = "top"
  )

```

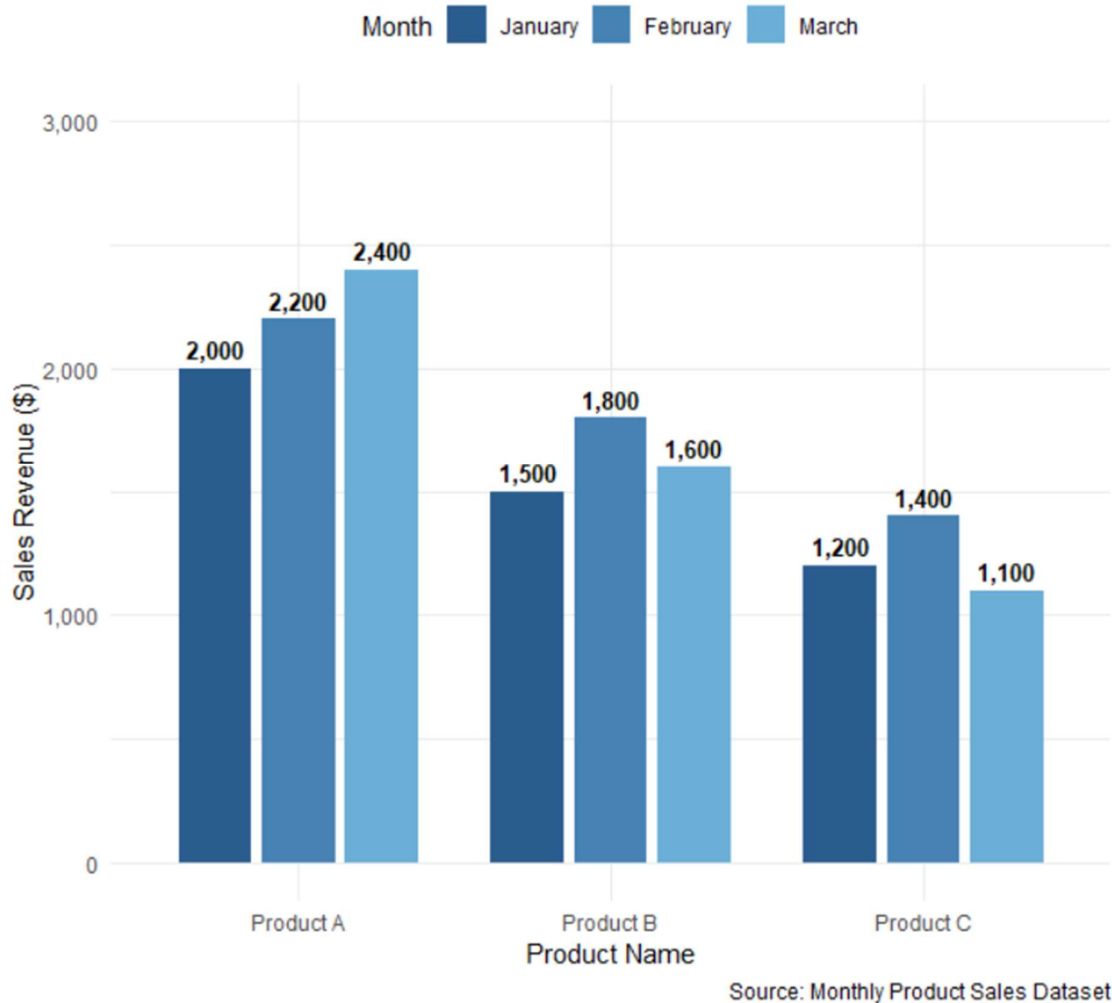
5. Display Chart

```
print(grouped_bar)
```

Output:

Q1 Product Sales Comparison

Monthly sales breakdown for Product A, B, and C



2. Generate a stacked area chart to represent the overall sales trend for all products over the three months.

Code:

```
# =====  
# TASK 2: STACKED AREA CHART  
# =====  
  
# 1. Load Required Libraries  
if (!require("ggplot2")) install.packages("ggplot2")
```

```
if (!require("tidyr")) install.packages("tidyr")
```

```
library(ggplot2)
```

```
library(tidyr)
```

```
# 2. Define Dataset
```

```
sales_data <- data.frame(  
  Product_ID = c(1, 2, 3),  
  Product_Name = c("Product A", "Product B", "Product C"),  
  January = c(2000, 1500, 1200),  
  February = c(2200, 1800, 1400),  
  March = c(2400, 1600, 1100)  
)
```

```
# 3. Reshape Data to Long Format
```

```
sales_long <- pivot_longer(  
  sales_data,  
  cols = c("January", "February", "March"),  
  names_to = "Month",  
  values_to = "Sales"  
)
```

```
# Convert Month to an ordered factor and numeric index for continuous area plotting
```

```
sales_long$Month <- factor(sales_long$Month, levels = c("January", "February", "March"))  
sales_long$Month_Num <- as.numeric(sales_long$Month)
```

```
# 4. Create Stacked Area Chart
```

```
stacked_area <- ggplot(sales_long, aes(x = Month_Num, y = Sales, fill = Product_Name)) +  
  geom_area(alpha = 0.85, color = "white", linewidth = 0.5) +  
  scale_x_continuous(breaks = 1:3, labels = c("January", "February", "March")) +
```

```

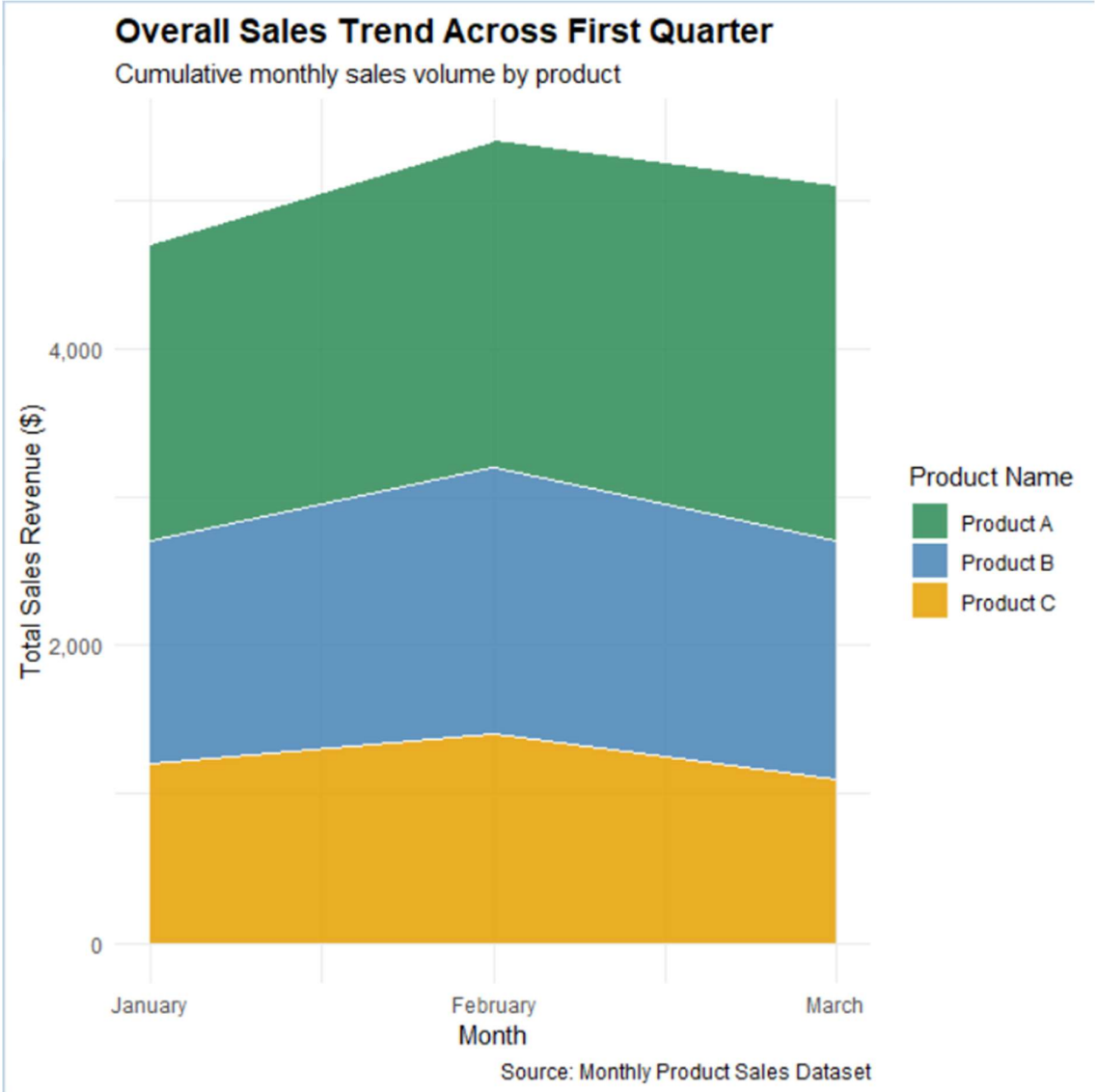
scale_y_continuous(labels = scales::comma) +
scale_fill_manual(values = c("Product A" = "#2E8B57", "Product B" = "#4682B4",
"Product C" = "#E69F00")) +
labs(
  title = "Overall Sales Trend Across First Quarter",
  subtitle = "Cumulative monthly sales volume by product",
  x = "Month",
  y = "Total Sales Revenue ($)",
  fill = "Product Name",
  caption = "Source: Monthly Product Sales Dataset"
) +
theme_minimal() +
theme(
  plot.title = element_text(face = "bold", size = 14),
  legend.position = "right"
)

```

5. Display Chart

```
print(stacked_area)
```


Output:



3. Build a table to show the monthly sales data for each product. Label the table elements.

Code:

```
# =====  
# TASK 3: DATA TABLE  
# =====  
  
# 1. Load Required Library  
if (!require("knitr")) install.packages("knitr")
```

```
library(knitr)
```

```
# 2. Define Dataset
```

```
sales_data <- data.frame(  
  Product_ID = c(1, 2, 3),  
  Product_Name = c("Product A", "Product B", "Product C"),  
  January = c(2000, 1500, 1200),  
  February = c(2200, 1800, 1400),  
  March = c(2400, 1600, 1100)  
)
```

```
# 3. Calculate Q1 Total Sales
```

```
sales_table <- sales_data  
sales_table$`Total Sales` <- rowSums(sales_table[, c("January", "February", "March")])
```

```
# 4. Format Column Labels
```

```
colnames(sales_table) <- c(  
  "Product ID",  
  "Product Name",  
  "January Sales ($)",  
  "February Sales ($)",  
  "March Sales ($)",  
  "Q1 Total Sales ($)"  
)
```

```
# 5. Render Table
```

```
kable(  
  sales_table,  
  align = c("c", "l", "r", "r", "r", "r"),  
  caption = "Table 1: Monthly Product Sales Summary (Q1)"
```

)

Output:

Table: Table 1: Monthly Product Sales Summary (Q1)

Product ID	Product Name	January Sales (\$)	February Sales (\$)	March Sales (\$)	Q1 Total Sales (\$)
1	Product A	2000	2200	2400	6600
2	Product B	1500	1800	1600	4900
3	Product C	1200	1400	1100	3700